FACULTY TECHNICAL REPORT • COMPUTER SCIENCE & SOFTWARE ENGINEERING

Computational & Software Engineering Architecture of the Textile Bioreactor Digital Twin

Algorithms, Numerical ODE Solvers, 3D WebGL Graphics, Reactive State Architecture & Distributed Microservices

Live Digital Twin: https://textile-bioreactor-digital-twin.vercel.app

Backend API: https://textile-bioreactor-digital-twin-api.onrender.com

Repository: sarimrafio/CRE_Project (GitHub)

Frameworks: React 19, Three.js, WebGL/GLSL, Python FastAPI, SciPy, Vite

1. Executive Summary for Academic Faculty

CORE COMPUTATIONAL CONTRIBUTIONS

This project translates Chemical Reaction Engineering (CRE) mass balances into a photorealistic, real-time industrial software system using **five foundational Computer Science pillars**:

1. **Numerical Analysis & Algorithms:** Client-side explicit **4th-Order Runge-Kutta (RK4)** vector integrator solving 3-state non-linear coupled differential equations in **<3.5 ms** at 60 FPS, combined with **Simpson's 1/3 Composite Rule** numerical quadrature.
2. **Computer Graphics & WebGL:** Custom procedural **GLSL Fragment/Vertex Shaders** computing real-time wave harmonics and dynamic color mixing on the GPU, paired with parametric 3D mesh construction.
3. **Reactive State Management:** Functional state pipelines orchestrated via a **Directed Acyclic Graph (DAG)** in React 19, preventing render waterfalls across 9 live sliders.
4. **Distributed Microservices:** Asynchronous **FastAPI / Uvicorn (ASGI)** REST API executing SciPy stiff solvers, backed by an offline-first client failover engine.
5. **DevOps & Headless Automation:** Automated **Playwright / Chromium** compilation pipeline rendering KaTeX mathematical proofs into A4 PDFs, deployed on **Vercel Edge CDN**.

2. Numerical Analysis & Scientific Computing Algorithms

A. EXPLICIT 4TH-ORDER RUNGE-KUTTA (RK4) SPATIAL SOLVER

To eliminate network latency during interactive parameter manipulation, the continuous spatial ordinary differential equations (ODEs) describing substrate S, biomass X_b, and dissolved oxygen $extDO$ along the reactor length are discretized and solved locally in JavaScript:

Ξec{y}_{n+1} = Ξec{y}_n + Ξrac{\Delta au}{6}\left(Ξec{k}_1 + 2Ξec{k}_2 + 2Ξec{k}_3 + Ξec{k}_4 ight)

- **Discretization Scale:** Evaluates $N = 100$ spatial slices across 3 coupled equations (300 calculations per frame).
- **Time Complexity:** $O(N \cdot M)$ where $M = 3$. Execution benchmark is $2.8ext{--}3.4extms$ on consumer CPUs.
- **Coupled Equations Solved:** ↟rac{dS}{d au} = -↟rac{\mu(S, ext{DO})}{Y}X_b, \quad ↟rac{dX_b}{d au} = (\mu - k_d)X_b, \quad ↟rac{d(ext{DO})}{d au} = k_La(ext{DO}_{ ext{sat}} - ext{DO}) - ext{OUR}(z)

B. COMPOSITE SIMPSON'S 1/3 RULE QUADRATURE

Because the reciprocal Monod-Haldane rate law f(S) = \frac{1}{-r_s(S)} = \frac{K_S + S + S^2/K_I}{(\mu_{\max}/Y)X_b S \cdot \frac{ ext{DO}}{K_{OH}+ ext{DO}}} cannot be integrated analytically, the Levenspiel PFR volume is calculated via numerical quadrature across $n = 200$ subintervals with fourth-order truncation error $O(h^4)$:

V_{ ext{PFR}} = Q_0 \int_{S_e}^{S_0} \frac{1}{-r_s(S)}\,dS \approx Q_0 \frac{h}{3} \left[f(S_0) + 4\sum_{j=1}^{n/2} f(S_{2j-1}) + 2\sum_{j=1}^{n/2-1} f(S_{2j}) + f(S_n) ight]

C. TRANSCRITICAL BIFURCATION ROOT-FINDING

An exact analytical root finder solves for the critical biological washout threshold heta_{c,\min} = \frac{1}{\mu(S_0, ext{DO}) - k_d}. When the user reduces sludge retention time $heta_c \leq heta_{c,\min}$, the mathematical engine detects the eigenvalue sign change, triggering biomass collapse ($X_b o 0$) in real time.

3. Computer Graphics & WebGL Pipeline (Three.js & GLSL)

A. GPU FRAGMENT SHADERS (CUSTOM GLSL)

Rather than using static textures or pre-rendered animations, fluid movement in open basins and transparent pipes is rendered directly on the GPU via custom OpenGL Shading Language (GLSL) code:

- **Wave Harmonics:** Generates fluid surface motion using real-time trigonometric wave interference: ext{Caustic}(\Ξec{u}, t) = \sin(k_1 u_x + \omega_1 t) \cdot \cos(k_2 u_y - \omega_2 t) + \sin(k_3 (u_x + u_y) + \omega_3 t)
- **Dynamic Color Mixing:** The shader samples the mathematical conversion $X = (S_0 - S_e)/S_0$ and smoothly interpolates fragment colors between raw indigo dye ($extRGB : [0.11, 0.13, 0.28]$) and purified emerald effluent ($extRGB : [0.18, 0.44, 0.32]$).

B. PROCEDURAL PARAMETRIC 3D GEOMETRY

The 3D environment does not swap fixed CAD files. All reactor geometries are computed parametrically from physical dimensions:

- **CSTR Vessel:** Re-computes radius R = \sqrt{\rac{V}{\pi H}} and recalculates cylinder vertices and liquid surfaces dynamically.
- **PFR Serpentine Basin:** Evaluates total channel length L = \rac{V}{W \cdot H} and extrudes a 3D spline curve (`CatmullRomCurve3`) creating four 180° rounded baffle U-turns.

C. PHYSICALLY BASED RENDERING (PBR) MATERIALS

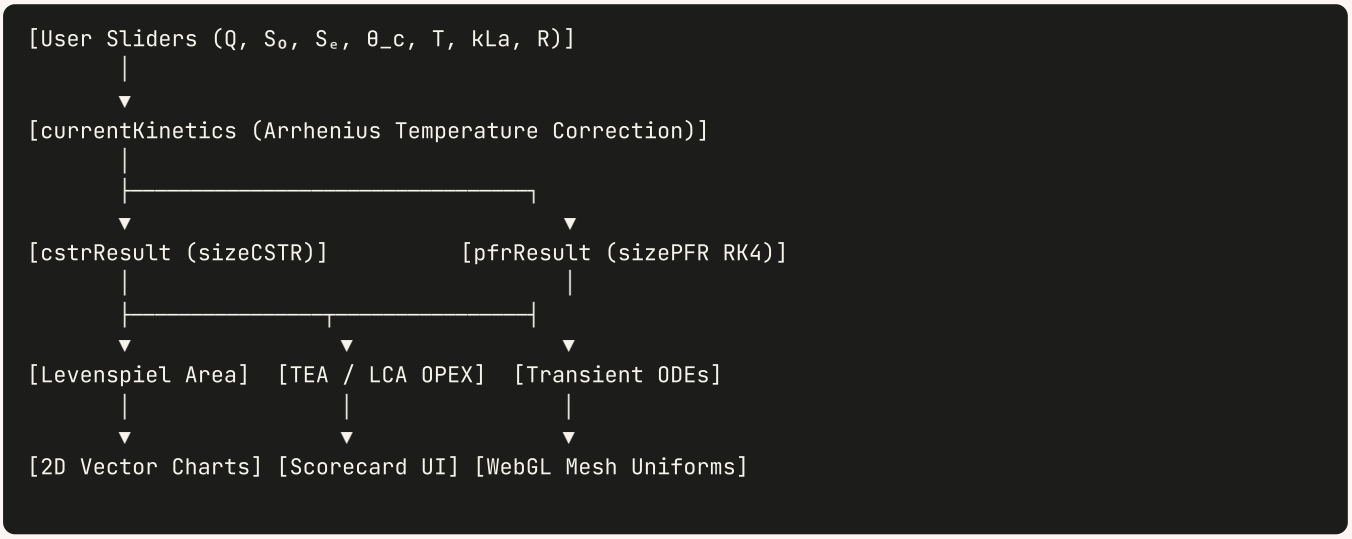
Employs the Cook-Torrance microfacet specular reflection model for realistic lighting:

- Reinforced concrete basin walls: Roughness = 0.88, Metalness = 0.05.
- Galvanized steel catwalks and air headers: Roughness = 0.22, Metalness = 0.95.
- Transparent borosilicate glass pipes: Transmission = 0.92, Index of Refraction (IOR) = 1.52.

4. Reactive Systems Architecture & Dependency DAG

DIRECTED ACYCLIC DEPENDENCY GRAPH (DAG)

To eliminate UI stutter and render waterfalls across 9 simultaneous sliders, state evaluation follows a strict functional DAG implemented via React 19 memoization:



- **Functional Purity:** Solvers are written as pure functions (\Ξec{p}) o \Ξec{r} with zero global state mutation.

- **Subtree Isolation:** Moving the Virtual Sonde probe slider (z/L) updates only the 2D tooltip coordinate overlay; the computationally heavy 3D WebGL scene graph skips re-rendering entirely.

5. Distributed Systems & Asynchronous Microservices

FASTAPI COMPUTE WORKER & OFFLINE-FIRST FAILOVER

- **Non-Blocking Event Loop:** The backend is written in Python 3.11 using **FastAPI** and **Uvicorn (ASGI)** for high-throughput asynchronous request handling.
- **Contract-First Serialization:** Employs **Pydantic v2** data models to enforce scientific bounds (e.g., verifying $S_e < S_0$ and $heta_c > 0$ with automatic 422 HTTP responses).
- **Stiff ODE Verification:** Executes SciPy's `solve_ivp` with the `Radau` implicit Runge-Kutta solver to verify numerical stability against client-side RK4.
- **Fault-Tolerant Local Fallback:** If the cloud microservice is sleeping (cold start) or unreachable, the frontend automatically falls back to its local JavaScript RK4 engine with zero UI freezing (`backendStatus: 'local_fallback'`).

6. Automated Headless Browser Automation (Playwright CI/CD)

HEADLESS CHROMIUM VECTOR PDF COMPILATION PIPELINE

To produce publication-ready PDF documentation directly from source code, we designed an automated headless browser compilation pipeline:

1. **Mathematical Typography Engine:** KaTeX converts 368 LaTeX mathematical formulas into client-side vector MathML and SVG elements with crisp sub-pixel rendering.
2. **CSS Paged Media Formatting:** Strict print styling using `@page { size: A4 portrait; margin: 14mm 12mm; }` enforces page break rules, margin boxes, and dynamic running page counters.
3. **Playwright Execution:** A Node.js orchestration script launches headless Chromium, navigates to the compiled DOM, verifies full math rendering, and outputs vector PDFs via `page.pdf()`.

7. Full Computer Science Technology Stack Summary

CS Discipline	Technologies / Libraries	Technical Implementation in Project
Numerical Algorithms	JavaScript ES2023, SciPy	In-browser RK4 vector solver, Simpson's 1/3 quadrature, transcritical bifurcation.
Computer Graphics	WebGL, Three.js, React Three Fiber	Real-time 3D plant rendering, scene graphs, orbit controls, procedural meshes.
GPU Shader Coding	GLSL (OpenGL ES 3.0)	Custom procedural fluid wave harmonics & dynamic chemical color grading.
Reactive Systems	React 19, Tailwind CSS	Virtual DOM reconciliation, memoized dependency DAG, zero-latency slider UI.
Distributed Systems	Python 3.11, FastAPI, Uvicorn	Asynchronous REST microservice for scientific ODE verification.
Data Validation	Pydantic v2, JSON Schema	Strict scientific boundaries, contract enforcement, and serialization.
Browser Automation	Playwright, Headless Chromium	Automated test verification and vector PDF handbook compilation.
Build & Cloud Edge	Vite 8, Rollup, Vercel Edge, Docker	Module tree-shaking, code splitting (311 KB gzipped bundle), global edge CDN.

Live Production URL: https://textile-bioreactor-digital-twin.vercel.app | **Source Code:** GitHub: sarimrafio/CRE_Project